Python Activity-Based Learning

Case Study 2: Progressive Electricity Billing

1. Problem Statement

Calculate a household electricity bill using progressive consumption slabs, a fixed charge, a surcharge, and a rebate. Compare bills for several months and flag an unusual increase.

2. Objectives

- Calculate energy charges using progressive slabs.
- Add fixed charges and a surcharge, then apply a qualifying rebate.
- Compare monthly bills and flag an increase above a chosen percentage.

3. Assumptions

- First 100 units cost ₹2 per unit; units 101–200 cost ₹3 per unit; units above 200 cost ₹5 per unit.
- Fixed charge is ₹50. A 5% surcharge is applied to energy charges only.
- A 10% rebate on the subtotal (energy + fixed charge + surcharge) is given when consumption is at most 100 units.
- An increase greater than 30% from the previous month is unusual.

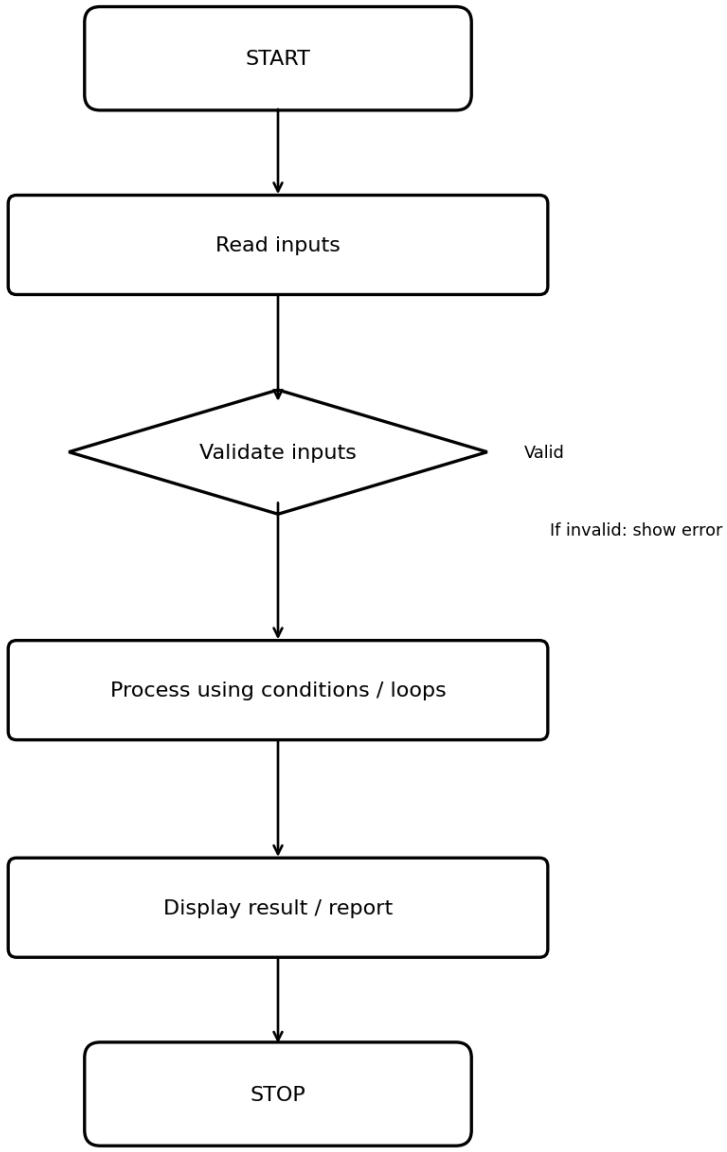
4. Algorithm

1. Start.
2. Read the number of months and each month's consumption.
3. Validate that consumption is not negative.
4. Calculate slab-wise energy charges using conditional statements.
5. Add fixed charge and surcharge; apply rebate if eligible.
6. Store each bill in a list and compare consecutive bills.
7. Display itemized bills and any unusual increase.
8. Stop.

5. Flowchart

The diagram below shows the overall program flow. The validation decision represents the input checks described in the algorithm.

Flowchart: Progressive Electricity Billing



6. Pseudocode

```
START
READ number_of_months
CREATE empty list bills
FOR each month
    READ units
    IF units < 0 THEN
        DISPLAY invalid input
    ELSE
        CALCULATE slab-wise energy_charge
        surcharge = 5% of energy_charge
        subtotal = energy_charge + 50 + surcharge
        IF units <= 100 THEN subtotal = subtotal - 10% of subtotal
        APPEND subtotal to bills
    END IF
```

```
END FOR
FOR each bill after the first
    increase = (current - previous) / previous * 100
    IF increase > 30 THEN DISPLAY unusual increase
END FOR
DISPLAY bills
STOP
```

7. Python Source Code

The solution uses only basic Python features from the assigned syllabus.

```
months = int(input("Enter number of months: "))
bills = []

for month in range(1, months + 1):
    units = int(input(f"Units used in month {month}: "))

    if units < 0:
        print("Invalid units. Enter a non-negative value.")
        continue

    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    surcharge = energy * 0.05
    subtotal = energy + 50 + surcharge
    rebate = 0

    if units <= 100:
        rebate = subtotal * 0.10

    bill = subtotal - rebate
    bills.append(bill)
    print(f"Month {month}: Energy=₹{energy:.2f}, "
          f"Surcharge=₹{surcharge:.2f}, Rebate=₹{rebate:.2f}, "
          f"Bill=₹{bill:.2f}")

for i in range(1, len(bills)):
    if bills[i - 1] > 0:
        increase = (bills[i] - bills[i - 1]) / bills[i - 1] * 100
        if increase > 30:
            print(f"Unusual increase in month {i + 1}: {increase:.1f}%")

if not bills:
    print("No valid bills were calculated.")
```

8. Test Cases

Test input / condition	Expected result	Purpose
100, 250	100 units; 250 units	First bill receives rebate; second uses all three slabs
150, 200	150 units; 200 units	Bills calculated with second slab

-5	Negative units	Invalid input message

9. Sample Output Screenshot

Sample console output for one test run:

```
C:\Users\Badri\PycharmProjects\PythonProject-1\.venv\Scripts\python.exe "C:\Users\Badri\PycharmProjects\PythonProject-1\Case Studies\case
 study - 4.py"
Enter number of months: 2
Units used in month 1: 10000
Month 1: Energy=₹49500.00, Surcharge=₹2475.00, Rebate=₹0.00, Bill=₹52025.00
Units used in month 2: 20000
Month 2: Energy=₹99500.00, Surcharge=₹4975.00, Rebate=₹0.00, Bill=₹104525.00
Unusual increase in month 2: 100.9%

Process finished with exit code 0
```

10. Conclusion

The program applies progressive slabs and conditional rebates, stores bills in a list, and uses a loop to identify unusually large month-to-month increases.

Note: Assumed rates, thresholds, and sample values are for this educational exercise and can be changed if the instructor supplies different rules.